Top 100 SQL Interview Q&A (Data Analytics) — 2025 | Part 1: Questions 1–25

> **Dialect locked: MySQL 8+ (Workbench). Style rules:** Keep queries simple (SELECT/WHERE/JOIN/GROUP BY/HAVING/ORDER BY, basic windows like ROW_NUMBER/RANK/DENSE_RANK/LAG, pivots with CASE). Every question includes: **Difficulty · Comprehensive Question · Hint (+mini function recap) · Sample Table(s)** *(unique to this question)* **· Query · Steps · Final Answer · Easy Explanation**.

---

1) 2nd Highest Salary per Department (Google) — Difficulty: Easy–Medium

Comprehensive Question: From Google HR, show the **second-highest distinct** salary per department.

Hint: Rank distinct salaries within each department, keep rank = 2.\ **Function recap:** `DENSE_RANK()` gives the same rank to ties without gaps (1,2,2,3).

Sample table — ``

emp_id	dept_id	salary
1	ENG	5000
2	ENG	7000
3	ENG	7000
4	MKT	4000
5	MKT	3000

Query (MySQL):

```
WITH r AS (
  SELECT dept_id,
         salary,
         DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM employees_q1
)
SELECT dept_id, salary AS second_highest_salary
FROM r
WHERE rnk = 2;
```

Steps: Partition by dept → order salary desc → dense_rank → filter rnk=2.\ **Final Answer:** ENG → 5000; MKT → 3000.\ **Easy Explanation:** Line salaries biggest→smallest per dept; pick the 2nd unique value.

---

2) DAU, 7-Day Rolling Avg & MAU (Meta) — Difficulty: Medium

Comprehensive Question: Show Daily Active Users, 7-day rolling average of DAU, and Monthly Active Users.

Hint: Count distinct users per day; average last 7 days; count distinct users in that month.\ **Function recap:** `AVG(x) OVER(ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)` averages the current + previous 6 rows.

Sample table — ``

user_id	event_ts
10	2025-07-01 10:00:00
11	2025-07-01 11:00:00
10	2025-07-02 09:00:00
12	2025-07-03 08:00:00

Query (MySQL):

```
WITH daily AS (
  SELECT DATE(event_ts) AS day,
         COUNT(DISTINCT user_id) AS dau
  FROM events_q2
  GROUP BY DATE(event_ts)
)
SELECT d.day,
       d.dau,
       AVG(d.dau) OVER (
         ORDER BY d.day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS dau_7day_avg,
       (
         SELECT COUNT(DISTINCT e2.user_id)
         FROM events_q2 e2
         WHERE YEAR(e2.event_ts)=YEAR(d.day)
           AND MONTH(e2.event_ts)=MONTH(d.day)
       ) AS mau
FROM daily d
ORDER BY d.day;
```

Steps: Daily DAU → 7-day rolling avg → monthly distinct via subquery.\ **Final Answer:** Table with day, DAU, 7-day avg, MAU.\ **Easy Explanation:** Count daily users, smooth over a week, also show monthly uniques.

---

3) Customers With Purchases in 3 Consecutive Months (Amazon) — Difficulty: Medium

Hint: Turn orders into monthly buckets, number months per customer, use the `(month_num - row_num)` trick to detect runs.\ **Function recap:** `ROW_NUMBER()` restarts numbering per customer.

Sample table — ``

order_id	customer_id	order_date
101	C1	2025-01-15
102	C1	2025-02-20
103	C1	2025-03-05
104	C2	2025-01-10
105	C2	2025-03-12

Query (MySQL):

```
WITH months AS (
  SELECT DISTINCT customer_id,
         DATE_FORMAT(order_date,'%Y-%m-01') AS month_start
  FROM orders_q3
), num AS (
  SELECT customer_id,
         month_start,
         (YEAR(month_start)*12 + MONTH(month_start)) AS mnum,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY month_start)
AS rn
  FROM months
)
SELECT customer_id
FROM num
GROUP BY customer_id
HAVING MAX(mnum - rn) - MIN(mnum - rn) >= 2;
```

Steps: Distinct customer-months → month number & row_number → range check ≥2.\ **Final Answer:** C1.\ **Easy Explanation:** Found a Jan-Feb-Mar streak.

---

4) Top-3 Products by Revenue per Category (Walmart) — Difficulty: Easy–Medium

Hint: Sum `qty*price` per product; rank within category; keep top 3.\ **Function recap:** `DENSE_RANK()` handles ties cleanly.

Sample tables — `**, **` `products_q4`

product_id	category
P1	Toys
P2	Toys
P3	Toys
P4	Games

`order_items_q4`

product_id	qty	price
P1	10	20
P2	5	40
P3	3	60

Query (MySQL):

```
WITH revenue AS (
  SELECT p.category,
         oi.product_id,
         SUM(oi.qty*oi.price) AS rev
  FROM order_items_q4 oi
  JOIN products_q4 p ON p.product_id=oi.product_id
  GROUP BY p.category, oi.product_id
), ranked AS (
  SELECT category, product_id, rev,
         DENSE_RANK() OVER (PARTITION BY category ORDER BY rev DESC) AS rnk
  FROM revenue
)
SELECT category, product_id, rev
FROM ranked
WHERE rnk <= 3
ORDER BY category, rev DESC;
```

Steps: Compute revenue → rank per category → keep top 3.\ **Final Answer:** Top earners by category.\ **Easy Explanation:** Biggest money-makers per aisle.

---

5) Weekly Cohort Retention (Netflix) — Difficulty: Medium

Hint: Snap signup and activity to Monday; subtract weeks; count.\ **Function recap:** `WEEKDAY()` returns 0=Mon..6=Sun; `TIMESTAMPDIFF(WEEK, a, b)` gives week gaps.

Sample tables — `**, **` `users_q5`

user_id	signup_date
U1	2025-01-03
U2	2025-01-05

`events_q5`

user_id	event_ts
U1	2025-01-10 12:00:00
U1	2025-01-18 09:00:00
U2	2025-01-05 14:00:00

Query (MySQL):

```
WITH cohorts AS (
  SELECT user_id,
         DATE_SUB(signup_date, INTERVAL WEEKDAY(signup_date) DAY) AS
cohort_week
  FROM users_q5
), activity AS (
  SELECT e.user_id,
         DATE_SUB(DATE(e.event_ts), INTERVAL WEEKDAY(DATE(e.event_ts)) DAY)
AS active_week
  FROM events_q5 e
  GROUP BY e.user_id, active_week
), joined AS (
  SELECT c.cohort_week,
         a.active_week,
         TIMESTAMPDIFF(WEEK, c.cohort_week, a.active_week) AS wk_offset
  FROM cohorts c
  JOIN activity a ON a.user_id=c.user_id
)
SELECT cohort_week, wk_offset, COUNT(*) AS retained_users
FROM joined
WHERE wk_offset >= 0
GROUP BY cohort_week, wk_offset
ORDER BY cohort_week, wk_offset;
```

Steps: Week buckets → join → week offset → counts.\ **Final Answer:** Retention counts by cohort/week.\ **Easy Explanation:** Who came back 0,1,2... weeks after signup.

---

6) Detect Overlapping Subscriptions per User (Spotify) — Difficulty: Medium

Hint: Compare each start to the previous end per user.\ **Function recap:** `LAG(end_date)` peeks at the last row in order.

Sample table — ``

user_id	start_date	end_date
U1	2025-01-01	2025-02-01
U1	2025-01-15	2025-03-01
U2	2025-02-01	2025-03-01

Query (MySQL):

```
WITH s AS (
  SELECT user_id, start_date, end_date,
         LAG(end_date) OVER (PARTITION BY user_id ORDER BY start_date) AS
prev_end
  FROM subs_q6
)
SELECT DISTINCT user_id
FROM s
WHERE prev_end IS NOT NULL
  AND start_date < prev_end;
```

Steps: Window previous end → compare with current start → flag overlaps.\ **Final Answer:** U1.\ **Easy Explanation:** New plan began before previous ended.

---

7) Products With Older Sales but None in Last 30 Days (Target) — Difficulty: Medium

Hint: Create recent vs older product sets and anti-join.\ **Function recap:** Anti-join = left join + `IS NULL` on right side.

Sample table — ``

product_id	order_date
P1	2025-06-01
P2	2025-07-15

Query (MySQL):

```sql
SET @cutoff := DATE_SUB(CURDATE(), INTERVAL 30 DAY);
SELECT p.product_id
FROM (SELECT DISTINCT product_id FROM order_items_q7) p
LEFT JOIN (SELECT DISTINCT product_id FROM order_items_q7
           WHERE order_date >= @cutoff) r ON r.product_id=p.product_id
LEFT JOIN (SELECT DISTINCT product_id FROM order_items_q7
           WHERE order_date <  @cutoff) o ON o.product_id=p.product_id
WHERE r.product_id IS NULL
  AND o.product_id IS NOT NULL;
```

Steps: Cutoff → recent set → older set → keep old-but-not-recent.\ **Final Answer:** P1 (if today > 2025-07-01).\ **Easy Explanation:** Used to sell, went cold this month.

---

8) Running Monthly Cumulative Sales (Apple) — Difficulty: Easy

Hint: Reset the running sum each month.\ **Function recap:** `SUM(amount) OVER(PARTITION BY YEAR,MONTH ORDER BY date)`.

Sample table — ``

dt	amount
2025-07-01	100
2025-07-02	200
2025-07-03	50

Query (MySQL):

```sql
SELECT dt,
       SUM(amount) OVER (
         PARTITION BY YEAR(dt), MONTH(dt)
         ORDER BY dt ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS month_cume
FROM sales_q8
ORDER BY dt;
```

Steps: Partition by month → cumulative sum by day.\ **Final Answer:** 07-01→100; 07-02→300; 07-03→350.\ **Easy Explanation:** Month counter that keeps adding.

---

9) Users Whose 90th Percentile Session > 10 Minutes (YouTube) — Difficulty: Medium-Hard

Hint: Sort durations per user, pick the value at `CEIL(0.9 * count)`.\ **Function recap:** `ROW_NUMBER()` positions each session; `CEIL()` rounds up.

Sample table — ``

user_id	duration_seconds
U1	300
U1	900
U1	120

Query (MySQL):

```
WITH b AS (
  SELECT user_id,
         duration_seconds,
         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY duration_seconds)
AS rn,
         COUNT(*) OVER (PARTITION BY user_id) AS cnt
  FROM sessions_q9
)
SELECT user_id
FROM b
WHERE rn = CEIL(0.9*cnt)
  AND duration_seconds > 600;
```

Steps: Order per user → compute position → compare value > 600s.\ **Final Answer:** U1 (p90=900).\ **Easy Explanation:** Long-session threshold is above 10 minutes.

---

10) Deduplicate Profiles Keeping Latest Row (Airbnb) — Difficulty: Easy

Hint: Delete older duplicates per user by comparing timestamps.\ **Function recap:** Self-join to find a newer row for the same key.

Sample table — ``

user_id	email	updated_at
U1	a@x.com	2025-07-01 10:00:00
U1	a1@x.com	2025-07-03 09:00:00

Query (MySQL):

```
DELETE p1
FROM profiles_q10 p1
JOIN profiles_q10 p2
```

```
  ON p1.user_id=p2.user_id
 AND p2.updated_at>p1.updated_at;
```

Steps: Join older→newer → delete older rows.\ **Final Answer:** Keep the `2025-07-03 09:00:00` row.\ **Easy Explanation:** Leave only the freshest profile.

---

11) % of Orders that are Repeat Orders (Uber Eats) — Difficulty: Medium

Hint: Number orders per customer and mark those with order number > 1.\ **Function recap:** `ROW_NUMBER()` restarts per customer.

Sample table — ``

order_id	customer_id
1	C1
2	C2
3	C1
4	C3
5	C2

Query (MySQL):

```
WITH h AS (
  SELECT order_id,
         customer_id,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_id) AS
nth
  FROM orders_q11
)
SELECT ROUND(100 * SUM(nth>1) / COUNT(*), 2) AS pct_repeat_orders
FROM h;
```

Steps: Rank per customer → count nth>1 → divide by total.\ **Final Answer:** 40.00% on the sample.\ **Easy Explanation:** Share of orders that weren't someone's first.

---

12) Forward-Fill Last Known Non-Null Score per User (LinkedIn) — Difficulty: Medium

Hint: Create buckets that increase when a non-null appears; take `MAX(score)` within bucket.\ **Function recap:** `SUM(score IS NOT NULL)` acts like a running counter.

Sample table — ``

user_id	ts	score
U1	2025-07-01 10:00	5
U1	2025-07-01 11:00	NULL
U1	2025-07-01 12:00	NULL
U1	2025-07-01 13:00	7

Query (MySQL):

```
WITH g AS (
  SELECT user_id, ts, score,
         SUM(score IS NOT NULL) OVER (
           PARTITION BY user_id ORDER BY ts
         ) AS bucket
  FROM metrics_q12
)
SELECT user_id,
       ts,
       MAX(score) OVER (PARTITION BY user_id, bucket) AS score_ffill
FROM g
ORDER BY user_id, ts;
```

Steps: Running bucket per user → max within bucket → forward-filled score.\ **Final Answer:** 10:00→5, 11:00→5, 12:00→5, 13:00→7.\ **Easy Explanation:** Copy last known value down until a new one appears.

---

13) Pivot Category Sales into Columns (Starbucks) — Difficulty: Easy

Hint: Conditional sums to get one column per category.\ **Function recap:** `CASE WHEN … THEN amount ELSE 0 END` inside `SUM()`.

Sample table — ``

dt	category	amount
2025-07-01	coffee	120
2025-07-01	food	80
2025-07-01	merch	30

Query (MySQL):

```sql
SELECT dt,
       SUM(CASE WHEN category='coffee' THEN amount ELSE 0 END) AS coffee_amt,
       SUM(CASE WHEN category='food'   THEN amount ELSE 0 END) AS food_amt,
       SUM(CASE WHEN category NOT IN('coffee','food') THEN amount ELSE 0
END) AS other_amt
FROM sales_q13
GROUP BY dt
ORDER BY dt;
```

Steps: Group by date → 3 conditional sums.\ **Final Answer:** 2025-07-01 → coffee=120, food=80, other=30.\ **Easy Explanation:** Turn row categories into separate columns.

---

14) Time to First Purchase After Signup (Shopify) — Difficulty: Easy

Hint: Find each user's first order and subtract signup time.\ **Function recap:** `TIMESTAMPDIFF(MINUTE, a, b)` gives minutes between times.

Sample tables — `**, **` `users_q14`

user_id	signup_ts
U1	2025-07-01 09:00:00
U2	2025-07-02 10:00:00

`orders_q14`

order_id	user_id	order_ts
10	U1	2025-07-01 09:30:00
11	U1	2025-07-02 08:00:00

Query (MySQL):

```sql
WITH first_order AS (
  SELECT user_id, MIN(order_ts) AS first_ts
  FROM orders_q14
  GROUP BY user_id
)
SELECT u.user_id,
       TIMESTAMPDIFF(MINUTE, u.signup_ts, f.first_ts) AS minutes_to_first
FROM users_q14 u
JOIN first_order f ON f.user_id=u.user_id;
```

Steps: First order per user → join to users → minutes between times.\ **Final Answer:** U1 → 30; U2 excluded (no order).\ **Easy Explanation:** How long did a new signup take to buy.

---

15) A/B Test Conversion: Two-Proportion z-Score (Booking.com) — Difficulty: Medium-Hard

Hint: Compute conversions per variant and plug into the z-score formula.\ **Function recap:** Just aggregates and sqrt/division.

Sample table — ``

user_id	variant	converted
1	A	1
2	A	0
3	B	1
4	B	1

Query (MySQL):

```
WITH s AS (
  SELECT variant,
         COUNT(*) AS n,
         SUM(converted) AS x
  FROM ab_q15
  GROUP BY variant
), r AS (
  SELECT variant, n, x, x/n AS cr
  FROM s
)
SELECT a.cr AS cr_a,
       b.cr AS cr_b,
       (a.cr-b.cr)/SQRT(a.cr*(1-a.cr)/a.n + b.cr*(1-b.cr)/b.n) AS z_score
FROM r a
JOIN r b ON a.variant='A' AND b.variant='B';
```

Steps: Count visitors & conversions → compute rates → z-score.\ **Final Answer:** Returns CR for A & B and the z-score.\ **Easy Explanation:** Standard test to compare two conversion rates.

---

16) First Order Date per Customer (Flipkart) — Difficulty: Easy

Hint: Group by customer and take `MIN(order_date)`.\ **Function recap:** `MIN()` gives the earliest date per group.

Sample table — ``

order_id	customer_id	order_date
1	C1	2025-06-01
2	C1	2025-06-10
3	C2	2025-05-20

Query (MySQL):

```
SELECT customer_id,
       MIN(order_date) AS first_order_date
FROM orders_q16
GROUP BY customer_id;
```

Steps: Group by customer → min date.\ **Final Answer:** C1→2025-06-01; C2→2025-05-20.\ **Easy Explanation:** The first time each customer bought.

---

17) Customers with Multiple Orders on Same Day (Swiggy) — Difficulty: Easy

Hint: Count orders per customer per calendar date; keep counts ≥2.\ **Function recap:** `HAVING` filters on the aggregated result.

Sample table — ``

order_id	customer_id	order_ts
11	U1	2025-07-01 10:00:00
12	U1	2025-07-01 19:00:00
13	U2	2025-07-02 09:30:00

Query (MySQL):

```
SELECT DATE(order_ts) AS order_date,
       customer_id,
       COUNT(*) AS orders_that_day
FROM orders_q17
GROUP BY DATE(order_ts), customer_id
HAVING COUNT(*) >= 2
ORDER BY order_date, customer_id;
```

Steps: Convert to date → group by customer+date → filter ≥2.\ **Final Answer:** U1 on 2025-07-01.\ **Easy Explanation:** Who ordered more than once that day.

18) New vs Returning Users by Day (Zomato) — Difficulty: Medium

Hint: Find each user's first day, then compare their later days.\ **Function recap:** In MySQL, `SUM(condition)` treats true as 1, false as 0.

Sample table — ``

user_id	event_ts
A	2025-07-01 09:00:00
A	2025-07-02 10:00:00
B	2025-07-02 08:00:00

Query (MySQL):

```
WITH firsts AS (
  SELECT user_id, MIN(DATE(event_ts)) AS first_seen
  FROM events_q18
  GROUP BY user_id
), daily AS (
  SELECT DATE(event_ts) AS day, user_id
  FROM events_q18
)
SELECT d.day,
       SUM(d.user_id IN (SELECT user_id FROM firsts WHERE first_seen=d.day))
AS new_users,
       SUM(d.user_id IN (SELECT user_id FROM firsts WHERE
first_seen<>d.day)) AS returning_users
FROM daily d
GROUP BY d.day
ORDER BY d.day;
```

Steps: First day per user → user-day rows → count new vs returning.\ **Final Answer:** Daily counts of new and returning users.\ **Easy Explanation:** Compare today's visitors to their first day.

---

19) Active Subscriptions on a Given Date (Hotstar) — Difficulty: Easy

Hint: Active if started before (or on) date and not ended before date.\ **Function recap:** `IS NULL` catches open-ended subscriptions.

Sample table — ``

user_id	start_date	end_date
U1	2025-07-01	2025-07-30
U2	2025-06-20	NULL

Query (MySQL):

```
SET @d := '2025-07-15';
SELECT user_id
FROM subs_q19
WHERE start_date <= @d
  AND (end_date IS NULL OR end_date >= @d);
```

Steps: Pick a date → filter by start/end logic.\ **Final Answer:** U1, U2.\ **Easy Explanation:** Started earlier and hasn't ended yet.

---

20) Top-2 Products by Quantity per Day (BigBasket) — Difficulty: Medium

Hint: Rank by qty within each date, keep the first two.\ **Function recap:** `ROW_NUMBER()` gives 1,2,3… per date.

Sample table — ``

dt	product_id	qty
2025-07-01	P1	10
2025-07-01	P2	10
2025-07-01	P3	7

Query (MySQL):

```
WITH x AS (
  SELECT dt, product_id, qty,
         ROW_NUMBER() OVER (PARTITION BY dt ORDER BY qty DESC, product_id)
AS rn
  FROM sales_q20
)
SELECT dt, product_id, qty
FROM x
WHERE rn <= 2
ORDER BY dt, qty DESC, product_id;
```

Steps: Rank per day → filter top-2.\ **Final Answer:** For 2025-07-01 → P1(10), P2(10).\ **Easy Explanation:** The two biggest sellers each day.

21) Daily Revenue & Running Total (Myntra) — Difficulty: Easy

Hint: Sum by date, then accumulate over dates.\ **Function recap:** `SUM(revenue) OVER(ORDER BY date)` is a running total.

Sample table — ``

order_date	amount
2025-07-01	1000
2025-07-02	500
2025-07-03	700

Query (MySQL):

```
WITH d AS (
  SELECT order_date, SUM(amount) AS revenue
  FROM orders_q21
  GROUP BY order_date
)
SELECT order_date,
       revenue,
       SUM(revenue) OVER (ORDER BY order_date) AS running_total
FROM d
ORDER BY order_date;
```

Steps: Daily revenue → running sum.\ **Final Answer:** 7-1:1000, 7-2:1500, 7-3:2200.\ **Easy Explanation:** Add today's sales to yesterday's pile.

---

22) Simple Product Mix Pivot by Day (Reliance Retail) — Difficulty: Easy

Hint: Use three conditional sums: grocery, fashion, other.\ **Function recap:** `CASE` inside `SUM()` for pivoting.

Sample table — ``

dt	category	amount
2025-07-01	grocery	300
2025-07-01	fashion	200
2025-07-01	toys	50

Query (MySQL):

```
SELECT dt,
       SUM(CASE WHEN category='grocery' THEN amount ELSE 0 END) AS
grocery_amt,
       SUM(CASE WHEN category='fashion' THEN amount ELSE 0 END) AS
fashion_amt,
       SUM(CASE WHEN category NOT IN('grocery','fashion') THEN amount ELSE 0
END) AS other_amt
FROM txn_q22
GROUP BY dt
ORDER BY dt;
```

Steps: Group by date → 3 sums.\ **Final Answer:** 2025-07-01 → 300, 200, 50.\ **Easy Explanation:** Split one day's revenue into columns.

---

23) Customers Who Churned in Last 30 Days (Nykaa) — Difficulty: Medium

Hint: Build two sets (recent vs older buyers) and anti-join.\ **Function recap:** Anti-join = left join + `IS NULL`.

Sample table — ``

order_id	customer_id	order_date
1	C1	2025-05-01
2	C2	2025-07-05

Query (MySQL):

```
SET @cutoff := DATE_SUB(CURDATE(), INTERVAL 30 DAY);
WITH buyers AS (
  SELECT DISTINCT customer_id FROM orders_q23
), recent AS (
  SELECT DISTINCT customer_id FROM orders_q23 WHERE order_date >= @cutoff
)
SELECT b.customer_id
FROM buyers b
LEFT JOIN recent r ON r.customer_id=b.customer_id
WHERE r.customer_id IS NULL
  AND b.customer_id IN (
        SELECT customer_id FROM orders_q23 WHERE order_date < @cutoff
      );
```

Steps: Buyers set → recent buyers set → keep buyers not seen recently.\ **Final Answer:** C1 (if today > 2025-06-01).\ **Easy Explanation:** Bought before, silent this month.

---

24) De-duplicate Orders Keeping Latest Status (Meesho) — Difficulty: Easy

Hint: Delete any status row that has a newer sibling for the same order.\ **Function recap:** Self-join on `order_id` comparing timestamps.

Sample table — ``

order_id	status	updated_at
101	shipped	2025-07-01 10:00:00
101	delivered	2025-07-02 12:00:00

Query (MySQL):

```
DELETE os1
FROM order_status_q24 os1
JOIN order_status_q24 os2
  ON os1.order_id=os2.order_id
 AND os2.updated_at>os1.updated_at;
```

Steps: Find older rows → delete them.\ **Final Answer:** Keep only the delivered row.\ **Easy Explanation:** Keep the latest status per order.

---

25) Same-City Customer Pairs (Paytm, self-join) — Difficulty: Easy

Hint: Join the table to itself on city; exclude same IDs.\ **Function recap:** Self-join pairs rows within one table.

Sample table — ``

customer_id	name	city
1	Ana	Pune
2	Ben	Pune
3	Cai	Mumbai

Query (MySQL):

```sql
SELECT a.customer_id AS cust1,
       b.customer_id AS cust2,
       a.city
FROM customers_q25 a
JOIN customers_q25 b
  ON a.city=b.city
 AND a.customer_id<>b.customer_id
ORDER BY a.city, cust1, cust2;
-- If you want unique pairs only, add: WHERE a.customer_id < b.customer_id
```

Steps: Self-join on city → remove self matches → (optionally) keep unique pairs.\ **Final Answer:** (1,2,Pune) and (2,1,Pune).\ **Easy Explanation:** Match people who live in the same city.

> ✅ Updated **Questions 1–25** so **each has its own unique sample table** (e.g., `employees_q1`, `events_q2`, … `customers_q25`) and queries reference those tables directly. If you want, I'll now apply the same per-question sample pattern to **Q26–Q50** next.